MINI DISTRIBUTED CLOUD PLATFORM

4-Week Collaborative Build Plan | 2 Developers | 3–4 hrs/day each

YOU (Person A)	FRIEND (Person B)	END-OF-DAY SYNC	GITHUB STRATEGY

HOW THIS PLAN WORKS

Every day is divided into parallel tracks — you and your friend work simultaneously on different parts. At end of day you sync for 30–45 minutes, walk each other through your code, and merge on GitHub. By end of Week 4 BOTH of you understand every single component deeply, not just your own track.

STEP 0 — BEFORE DAY 1: GitHub Repo & Environment Setup (Do Together, 1 hour)

REPO STRUCTURE TO SET UP ON DAY 0

mini-cloud/ master/ ← Master node FastAPI app api/ ← Route handlers scheduler/ ← Scheduling logic registry/ ← Node registry db/ ← Database models worker/ ← Worker node FastAPI app executor/ ← Docker task runner storage/ ← File chunk handler functions/ ← Lambda runner health/ ← Heartbeat reporter shared/ ← Pydantic models shared by both infra/ ← Docker-compose, configs, scripts tests/ ← Integration tests .github/ ← GitHub Actions CI (optional) README.md

NETWORK SETUP — WiFi NOW, WIRED LATER

Current setup: All 4 laptops on the same WiFi network. Future upgrade: Switch to wired Ethernet (plug into a switch) when you want lower latency for stress testing. What changes when you go wired: Nothing in the code. Just reassign static IPs if they change, re-run ping tests, and tune heartbeat timeout back from 20s → 15s. WiFi tips for now: - Assign static IPs via DHCP reservation in your router (not manual — router restarts will break manual assignments) - Keep all 4 laptops on the 5GHz band if your router supports it (lower interference) - During stress tests on WiFi, expect higher latency variance — this is normal, not a bug - To simulate node failure: kill the worker process (do NOT rely on disconnecting WiFi — too unpredictable for testing)

GITHUB BRANCH & MERGE STRATEGY

main	Always working, always deployable. Merge only at end of each week after testing.	Both — protected
dev	Integration branch. Both push feature branches here first, then	Both

	main.	
feat/A-*	Your personal feature branches. Example: feat/A-heartbeat-api	You (Person A)
feat/B-*	Friend's feature branches. Example: feat/B-heartbeat-worker	Friend (Person B)
fix/*	Bug fix branches either person creates during sync.	Both

DAILY WORKFLOW RITUAL

1. Morning: git pull origin dev (get latest) 2. Work on your feat/* branch all day 3. Evening sync: both open PRs into dev, review each other's code for 20 min, merge 4. Weekly: merge dev into main after full test

WEEK 1 — FOUNDATION: Node Registration, Heartbeat & Project Skeleton (Days 1–7)

WEEK 1 GOAL

By Sunday of Week 1: Master node is running. All 3 worker nodes have registered themselves with the master. Heartbeats are flowing every 5 seconds. Dead node detection is working (kill the worker process or disconnect from WiFi — master detects it within 15–20 seconds). Both of you understand the ENTIRE week-1 codebase.

DAY 1 — Project skeleton + shared data models + FastAPI boilerplate on both nodes

PREREQUISITES BEFORE THIS DAY

Python 3.10+ installed on all 4 machines. Docker Engine running. Git configured with SSH keys. Static IPs assigned (set as DHCP reservations in your router). All 4 machines connected to the same WiFi network and can ping each other — verify: ping 192.168.1.11 from master. NOTE: you are starting on WiFi; plan to switch to wired Ethernet later for stress testing — no code changes needed when you do.

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Set up master/ folder structure. Create master/main.py with bare FastAPI app. Add GET /health endpoint returning {status: ok, node: master}. Run it: uvicorn master.main:app --host 0.0.0.0 --port 8000	Set up worker/ folder structure. Create worker/main.py with bare FastAPI app. Add GET /health endpoint returning {status: ok, node: worker, ip: <machine_ip>}. Run it: uvicorn worker.main:app --host 0.0.0.0 --port 8001	*Both create branches: feat/A-skeleton and feat/B-skeleton. At EOD: both PR into dev. Review structure together — folders must mirror the agreed layout. Merge both. This is the one time you set up the repo skeleton together.*
Create shared/models.py with Pydantic models: - NodeInfo(id, ip, port, cpu_percent, ram_percent, active_tasks, status) - TaskRequest(task_id, image, command, cpu_limit, ram_limit) - TaskResult(task_id, status, output, error, worker_id) - HeartbeatPayload(node_id, ip, cpu_percent, ram_percent, active_tasks, timestamp)	Write unit tests for every Pydantic model in tests/test_models.py. Try creating model instances with invalid data (wrong types, missing fields) to verify validation. Also: create infra/docker-compose.yml with just master and one worker service (no logic yet — just confirming containers start).	*shared/ is a dependency of BOTH master and worker. Person A owns the models file. Person B's tests act as the contract — if tests pass, the models are correct. Branch: feat/A-models and feat/B-model-tests. PR both into dev.*

END-OF-DAY SYNC AGENDA

1. Walk through each other's folder structure — can you explain why each folder exists?
2. Read Person B's model tests out loud and verify they match Person A's model definitions exactly
3. Confirm all 4 laptops can reach master:8000/health and worker:8001/health over WiFi — run: curl http://192.168.1.10:8000/health from each machine

4. Discuss: What does NodeInfo.status represent? What values can it have? (Define: 'healthy', 'degraded', 'dead')
5. Set up tomorrow's branches: feat/A-registry and feat/B-heartbeat-worker

DAY 2 — Node Registry on master + Worker self-registration on startup

PREREQUISITES BEFORE THIS DAY

Day 1 merged into dev. shared/models.py is available to both master and worker. All /health endpoints reachable across machines over WiFi.

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Create master/registry/node_registry.py: - In-memory dict: nodes = {} (key=node_id, value=NodeInfo) - add_node(node_info: NodeInfo) - remove_node(node_id: str) - get_all_nodes() -> list[NodeInfo] - get_healthy_nodes() -> list[NodeInfo] - update_node(node_id, heartbeat: HeartbeatPayload) Create master/api/registry_routes.py: - POST /nodes/register (worker calls this at startup) - GET /nodes (list all registered nodes) - GET /nodes/{node_id} (get single node info)	Create worker/health/registrar.py: - On worker startup, call POST master_ip:8000/nodes/register with its own NodeInfo - Include: node_id (generate with uuid4), ip (auto-detect LAN IP using socket.getsockname), port, hostname - If registration fails (master unreachable), retry every 10 seconds - Log success: 'Registered with master as node-<id>' Test manually: start worker, watch master logs, call GET /nodes on master — your worker should appear.	*The interface contract: POST /nodes/register accepts NodeInfo (defined in shared/models.py). Person A writes the endpoint. Person B writes the client that calls it. They MUST use the same Pydantic model — no ad-hoc dicts. Branch: feat/A-registry / feat/B-registration. Integration test: Person B starts worker, Person A watches GET /nodes return it.*

END-OF-DAY SYNC AGENDA

1. Demo live: Person B starts a worker on their laptop. Person A opens GET /nodes on master — the worker must appear with correct IP, port, and status=healthy
2. Walk through the registration code line by line — Person A explains the registry dict, Person B explains the startup registrar
3. Discuss: What happens if the same worker restarts? Does it register twice? How do you handle duplicate node_ids? (Answer: use ip+port as unique key OR overwrite on re-register)
4. Discuss: Why is the registry in-memory right now instead of a database? What is the risk? (Answer: master restart wipes all nodes — we'll fix this in Week 2 with Redis)
5. Discuss: Why does the worker detect its own LAN IP instead of using 127.0.0.1? (Answer: 127.0.0.1 only routes to localhost — master needs the real LAN IP to reach the worker)

DAY 3 — Heartbeat system: workers ping master every 5s, master detects dead nodes

PREREQUISITES BEFORE THIS DAY

Node registration working end-to-end. GET /nodes returns registered workers. Both understand Day 2 code fully after sync.

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Create master/registry/health_monitor.py: - Background asyncio task (runs forever, checks every 5s) - For each node in registry: check if last_heartbeat timestamp is older than 15 seconds - If yes → set node.status = 'dead', log warning - If recovered → set node.status = 'healthy' Create master/api/heartbeat_routes.py: - POST /heartbeat (workers call this periodically) - Updates node's cpu_percent, ram_percent, active_tasks, last_heartbeat = datetime.now() Start health_monitor as a FastAPI lifespan background task using asyncio.create_task()	Create worker/health/heartbeat.py: - Async loop: every 5 seconds, collect real CPU% and RAM% using psutil library - Build HeartbeatPayload and POST it to master:8000/heartbeat - If POST fails, log error but keep trying (don't crash the worker) Install psutil: pip install psutil Test: print CPU and RAM locally before sending — verify the numbers look real. Also: start this heartbeat loop as a background task in worker startup (same pattern as master's health monitor — asyncio.create_task in lifespan).	*The heartbeat flow is the most critical operational flow in the whole system. Integration test (do together at EOD sync): Start master + 2 workers. Watch GET /nodes — both should show cpu/ram updating in real time. Then simulate node failure by killing the worker process (or disconnecting from WiFi). Within 15–20 seconds, that node's status should flip to 'dead' on the master. NOTE: on WiFi, set timeout to 20s (4 missed beats) to avoid false positives from momentary WiFi jitter. Branch: feat/A-health-monitor / feat/B-heartbeat-sender*

END-OF-DAY SYNC AGENDA

1. Live demo: 2 workers running, kill one worker process (or disconnect it from WiFi). Master should mark it dead within 15–20 seconds. Restart the process — it re-registers and goes healthy
2. Person B explains psutil: what does cpu_percent(interval=1) do? Why interval=1 and not 0?
3. Person A explains the asyncio lifespan pattern — why is a background task better than a thread here?
4. Discuss: What is the difference between 'dead' (heartbeat timeout) and 'unhealthy' (high CPU)? Should we treat them the same way for scheduling?
5. Discuss: WiFi introduces real latency jitter — what if a heartbeat is delayed by 3–4 seconds due to WiFi congestion but the worker is not actually dead? (Solution: use a missed-beat counter threshold of 4 instead of 3 while on WiFi; reduce back to 3 when you switch to wired)

DAY 4 — Redis for persistent node registry + structured logging foundation

PREREQUISITES BEFORE THIS DAY

Heartbeat system fully working. Both understand the full flow: startup → register → heartbeat → dead detection. Redis installed on master machine: sudo apt install redis-server && redis-cli ping (should return PONG).

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Add Redis persistence to node registry: - Install: pip install redis - In master/registry/node_registry.py, add Redis client - On add_node(): also HSET nodes:{node_id} with node data (serialize NodeInfo to JSON) - On startup: load existing nodes from Redis into in-memory dict (master restart recovery) - On remove_node() or mark dead: update Redis too This means if master restarts, it reads Redis and immediately knows about all nodes that were alive — no waiting for re-registration.	Create shared/logger.py — a structured logger both master and worker use: - Use Python's logging module with a JSON formatter - Every log line outputs: {timestamp, level, node_id, component, event, message, extra_data} - Example: {ts: '2024-01-01T10:00:00', level: 'INFO', node: 'worker-1', component: 'heartbeat', event: 'beat_sent', cpu: 45.2} - Replace all print() statements in both master and worker with this logger Why structured logs? So master can later ingest and query them (Week 4 monitoring).	*These are independent improvements — no integration needed today. But at EOD: Person A demos master restart recovery. Kill master, restart it, call GET /nodes — workers should already be there from Redis without re-registering. Person B demos structured log output — pipe worker logs to a file and inspect the JSON. Branch: feat/A-redis-registry / feat/B-structured-logging*

END-OF-DAY SYNC AGENDA
1. Master restart demo: kill master process, restart it, immediately call GET /nodes — workers appear without re-registering. Explain how Redis enabled this
2. Read a structured log line together — can you grep/filter it? Try: cat worker.log | python3 -c "import sys,json; [print(l) for l in sys.stdin if 'heartbeat' in l]"
3. Discuss: What is Redis? Why a key-value store and not PostgreSQL for the registry? (Answer: Redis is in-memory + extremely fast for frequently-written small data like heartbeats)
4. Discuss: What would happen if Redis goes down? Should we handle that? (Answer: yes, graceful degradation — fall back to in-memory only, log warning)
5. Review entire Week 1 codebase together. Each person explains the other person's code from scratch.

DAYS 5–6 — Buffer + Integration Testing + Week 1 Documentation

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Write integration test: tests/test_week1.py - Test 1: Start master, register 2 workers, verify GET /nodes returns both - Test 2: Send 5 heartbeats from a	Write tests/test_week1_worker.py: - Test 1: Worker startup triggers registration POST to master - Test 2: psutil returns valid CPU	*Both branches PR into dev, then dev into main — first real main merge! Code review: each person reviews the other's tests. Are edge cases covered? By EOD Day 6: any new developer should be able*

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
worker, verify cpu/ram updated on master - Test 3: Stop sending heartbeats, wait 20s, verify node status = 'dead' - Test 4: Kill master, restart, verify Redis restores node list Use httpx (async test client) for all HTTP calls.	and RAM floats between 0-100 - Test 3: Heartbeat payload matches HeartbeatPayload Pydantic model exactly - Test 4: Worker handles master being unreachable (should not crash, should retry) Also: write infra/README.md — step-by-step setup guide for running the full Week 1 system on 4 laptops.	*to follow infra/README.md and get the Week 1 system running from scratch on their machines.*

DAY 7 — Week 1 Knowledge Sync — Both Explain Everything

DAY 7 IS NOT A CODING DAY

This is a dedicated understanding day. No new features. Spend the full 3-4 hours going deep on everything built this week. The rule: each person must be able to explain every line of code the other person wrote. Use this day to fill any gaps.

PERSON A EXPLAINS TO FRIEND (30 min)

Walk through worker/health/registrar.py and worker/health/heartbeat.py line by line. Explain: socket.getsockname(), psutil.cpu_percent(), asyncio.create_task(), the retry loop logic, and why we use HeartbeatPayload instead of a raw dict.

FRIEND EXPLAINS TO YOU (30 min)

Walk through master/registry/node_registry.py and master/registry/health_monitor.py. Explain: the in-memory dict structure, the asyncio background loop, datetime comparison for timeout detection, Redis HSET serialization, and master restart recovery.

JOINT DISCUSSION (60 min)

Draw the full Week 1 data flow on paper together: Worker boots → registers → sends heartbeat → master updates registry → health monitor checks → dead detection → Redis persists state. For each arrow: what protocol? what data? what happens if it fails? If either of you cannot explain any part, fix it NOW before Week 2.

WEEK 2 — COMPUTE ENGINE: Task Scheduler + Docker Execution (Days 8–14)

WEEK 2 GOAL

By Sunday of Week 2: A user can POST a task (Docker image + command) to the master. The scheduler picks the best worker using resource-aware logic. The worker spins up a Docker container, runs the command, captures output, and returns it. Both of you can trace every step of this flow and explain every design decision.

DAY 8 — Task data models + task state machine + master task database

PREREQUISITES BEFORE THIS DAY

Week 1 fully working and merged to main. Both understand all Week 1 code. PostgreSQL installed on master: sudo apt install postgresql. Create database: createdb minicloud. Note: heartbeat timeout tuned to 20s for WiFi — remember to adjust back to 15s when switching to wired.

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Extend shared/models.py with task models: - TaskStatus enum: PENDING, SCHEDULED, RUNNING, COMPLETED, FAILED, CANCELLED - TaskRequest: task_id(auto uuid), image(str), command(str), cpu_limit(float=0.5), ram_limit_mb(int=256), priority(int=1) - TaskResult: task_id, worker_id, status, output(str), error(str), started_at, completed_at, duration_seconds - SchedulerDecision: task_id, chosen_worker_id, reason(str), score(float) Create master/db/models.py — SQLAlchemy ORM models mirroring the Pydantic models above. Create master/db/database.py — async SQLAlchemy engine, session factory, init_db() function.	Create master/db/task_repository.py: - create_task(task: TaskRequest) -> Task (inserts into DB, returns with generated ID) - get_task(task_id: str) -> Task - update_task_status(task_id, status, worker_id=None) - update_task_result(task_id, result: TaskResult) - get_tasks_by_status(status: TaskStatus) -> list[Task] - get_all_tasks(limit=50) -> list[Task] Write tests for every repository method in tests/test_task_repo.py using a test SQLite DB (not real Postgres — tests should be fast and isolated).	*This is a pure master-side day — no worker code changes. Branch: feat/A-task-models / feat/B-task-repository Integration point: Person A's Pydantic models are the input/output types of Person B's repository functions. If Person A changes a model field, Person B's tests will catch it immediately. Merge into dev at EOD.*

END-OF-DAY SYNC AGENDA

1. Explain the TaskStatus state machine: draw all valid transitions. Can a task go from COMPLETED back to RUNNING? From CANCELLED to PENDING? Why not?

2. Person B demos: insert a task, update its status to RUNNING, then COMPLETED — query and show the result

3. Discuss: Why async SQLAlchemy instead of sync? (Answer: FastAPI is async — sync DB calls block the event loop and prevent handling other requests during DB operations)

4. Discuss: Why keep TaskRequest and the ORM model separate? Why not use the ORM model directly in the API? (Answer: Pydantic validates input; ORM deals with DB — mixing them creates tight coupling)

DAY 9 — Resource-aware scheduler on master

PREREQUISITES BEFORE THIS DAY

Task DB and repository working. TaskStatus state machine agreed and documented. Both understand the ORM models.

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Create master/scheduler/scheduler.py — the core scheduling logic: score(node) = (1 - cpu_percent/100) * 0.5 + (1 - ram_percent/100) * 0.3 + (1 - active_tasks/MAX_TASKS) * 0.2 Higher score = better candidate. select_worker(task: TaskRequest) -> (NodeInfo, SchedulerDecision): 1. Get all healthy nodes from registry 2. If no healthy nodes → raise NoHealthyNodeError 3. Calculate score for each node 4. Pick highest score 5. Return (chosen_node, SchedulerDecision with reason='cpu:45%,ram:30%,tasks:1') Write unit tests: tests/test_scheduler.py - Test with 3 mock nodes of varying CPU/RAM - Verify the least-loaded node is always chosen - Test NoHealthyNodeError when all nodes are dead	Create master/api/task_routes.py: - POST /tasks — accepts TaskRequest, creates DB record with status=PENDING, returns {task_id, status: 'accepted'} - GET /tasks/{task_id} — returns current task status + result if completed - GET /tasks — list all tasks with their statuses - DELETE /tasks/{task_id} — cancel a PENDING task These routes do NOT execute tasks yet — they just accept and store them. Execution comes tomorrow. Wire the routes into master/main.py. Test manually with curl: curl -X POST http://master:8000/tasks \ -H 'Content-Type: application/json' \ -d '{"image": "alpine", "command": "echo hello"}'	*Scheduler (Person A) and API routes (Person B) come together tomorrow when the API calls the scheduler. Today they are independent. Branch: feat/A-scheduler / feat/B-task-api Critical: Person A must write scheduler tests without any API. Person B must write API tests without real scheduler — use a mock scheduler that always returns worker-1. Merge into dev at EOD.*

END-OF-DAY SYNC AGENDA

1. Person A explains the scoring formula — why these weights (0.5, 0.3, 0.2)? What happens if you change them?

2. Person B demos: POST a task, GET /tasks — see it in PENDING status. GET /tasks/{id} — see full detail

3. Together: run scheduler unit tests. Manually create 3 NodeInfo objects with different CPU/RAM and call select_worker — verify it picks the right one

4. Discuss: What if two tasks arrive at exactly the same millisecond and both choose the same worker? Is this a race condition? (Answer: yes — we solve it with a task lock in Week 2 Day 11)

5. Discuss: What should the API return if there are no healthy workers when a task is submitted? (Answer: 503 Service Unavailable with a clear message)

DAY 10 — Worker Docker executor — the most critical component

PREREQUISITES BEFORE THIS DAY

Docker SDK installed on worker machines: pip install docker. Test: python3 -c 'import docker; c = docker.from_env(); print(c.ping())' must return True. Scheduler and task API working on master.

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Wire task dispatch into master: - In POST /tasks route: after saving to DB, immediately call scheduler.select_worker() - POST the task to chosen_worker:8001/execute (async HTTP call using httpx) - Update task status to SCHEDULED with chosen worker_id - If worker POST fails → update status to FAILED, log error - The master POST to worker is async — do NOT wait for execution result here. Worker will call back. Create master/api/result_routes.py: - POST /tasks/{task_id}/result — workers call this when done - Updates DB with TaskResult, sets status=COMPLETED or FAILED	Create worker/executor/docker_runner.py — the heart of the compute engine: run_task(task: TaskRequest) -> TaskResult: client = docker.from_env() container = client.containers.run(image=task.image, command=task.command, detach=False, # wait for completion remove=True, # auto-delete container after mem_limit=f'{task.ram_limit_mb}m', nano_cpus=int(task.cpu_limit * 1e9), network_mode='none', # no internet access — security stdout=True, stderr=True) return TaskResult(output=container.decode(), status=COMPLETED) Create worker/api/executor_routes.py: - POST /execute — receives TaskRequest from master, runs docker_runner, POSTs result back to master - This must be async — use asyncio.create_task so the response to master returns immediately	*Today's integration is the biggest of the whole project. Full end-to-end test (do together during EOD sync): 1. POST task to master → master schedules → worker gets it → Docker runs → result POSTed back → GET /tasks/{id} shows COMPLETED with output Branch: feat/A-task-dispatch / feat/B-docker-executor Test with a simple image first: alpine with command 'echo hello world' Then test: python:3.11-slim with command 'python3 -c "print(2+2)"'*

END-OF-DAY SYNC AGENDA

1. Run the full end-to-end flow live. POST a task, watch logs on both master and worker, GET the result

2. Deliberately break it: submit a task with a non-existent Docker image (bad_image:latest) — what happens? Is the error handled gracefully?

3. Person B explains: what does detach=False mean? What is nano_cpus? Why network_mode='none'?
4. Person A explains: why does the worker use asyncio.create_task for execution? What would happen without it?
5. Discuss: Right now the worker executes tasks synchronously one at a time. What happens if 10 tasks arrive at once? (Preview: we address concurrency in Day 11)

DAY 11 — Concurrency, worker task queue, and failure handling

PREREQUISITES BEFORE THIS DAY

End-to-end task execution working. Both have seen a complete task lifecycle in the logs. Both understand the Docker SDK calls.

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Add a concurrency limit to worker: - Worker has a semaphore: asyncio.Semaphore(MAX_CONCURRENT=3) - Each /execute request acquires the semaphore before running Docker - If semaphore full → return 429 Too Many Requests to master - Master handles 429 → requeue task to another worker OR retry after 5s - Update active_tasks count in heartbeat: increment on task start, decrement on finish Also fix the race condition from Day 9: - Master uses an asyncio.Lock() around the scheduler select + task assign step - This prevents two simultaneous requests from both choosing the same worker	Create worker/executor/task_queue.py: - An asyncio.Queue(maxsize=10) on the worker - Instead of running Docker immediately in the route handler, PUT the task in the queue and return 202 Accepted immediately - A separate consumer coroutine reads from the queue and calls docker_runner.run_task() - This decouples receiving tasks from executing them Also handle task timeout: - Wrap docker_runner call with asyncio.wait_for(timeout=300) (5 minute max) - If timeout: kill the container (client.containers.get(id).kill()), return FAILED with error='timeout'	*These two improvements solve the same problem (concurrency) from different sides — master handles overflow, worker handles internal queuing. Branch: feat/A-scheduler-lock / feat/B-task-queue Integration test: submit 10 tasks simultaneously using a Python script with asyncio.gather(). Watch how they distribute across workers. Check that no task is lost.*

END-OF-DAY SYNC AGENDA

1. Run the 10-concurrent-task stress test. Observe: are tasks distributed fairly? Does any worker get overloaded?
2. Person A explains the asyncio.Lock — why is a lock needed when Python's GIL already prevents true parallelism in threads? (Answer: asyncio is cooperative — two coroutines can interleave between await points even in a single thread)
3. Person B explains asyncio.Semaphore vs asyncio.Queue — what is the difference? When would you use each?
4. Kill a worker mid-execution (ctrl+C). What happens to the running task? Does master detect it? Does the task stay

in RUNNING state forever? (Problem to solve: add task reaper in Week 3)

5. Discuss: Should the master retry a failed task automatically? How many times? What if the task itself is buggy and will always fail?

DAYS 12–13 — Task retry logic, orphan task reaper, and full Week 2 test suite

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Create master/scheduler/task_reaper.py: - Background asyncio task running every 30 seconds - Query DB for tasks in RUNNING status for more than 10 minutes - For each orphan: check if assigned worker is still alive in registry - If worker is dead → update task to FAILED with error='worker_died' - Optionally: if task.retry_count < 3, re-submit to scheduler as new task Also add retry tracking to TaskRequest: - Add retry_count(int=0), max_retries(int=3) fields - In task dispatch: if worker returns 503/429, increment retry_count, try next best worker	Write tests/test_week2_integration.py: - Test happy path: POST task → COMPLETED with correct output - Test bad image: POST task with garbage image → FAILED with docker error message - Test worker death: register worker, start task, kill worker process, verify reaper marks task FAILED - Test 10 concurrent tasks: verify all complete, verify active_tasks in heartbeat counts correctly - Test retry: mock a worker that fails twice then succeeds → verify task eventually COMPLETED Write tests/test_docker_runner.py with unit tests for docker_runner.py	*This is the hardest integration test week of the project. Run all tests on the actual multi-laptop setup, not mocks. Branch: feat/A-task-reaper / feat/B-week2-tests Goal: zero failing tests before merging dev → main on Day 14*

DAY 14 — Week 2 Knowledge Sync — Compute Engine Deep Dive

WEEK 2 UNDERSTANDING DAY

The compute engine is the most complex part of this project. Use this day to make sure both of you understand every component deeply before moving to storage.

JOINT EXERCISE: Trace a Task End-to-End

Pick a completed task from the DB. Together, trace EVERY step: 1. Which API route received the POST? 2. Which DB function created the record? 3. What did the scheduler score for each worker? 4. What HTTP call did master make to the worker? 5. What Docker SDK call ran the container? 6. How did the output get captured and returned? 7. Which DB function updated the final status? If you cannot answer any step, go find it in the code right now.

WEEK 3 — STORAGE + LAMBDA: Distributed Files + Function Execution (Days 15–21)

WEEK 3 GOAL

By Sunday of Week 3: Users can upload files and retrieve them even after a worker dies. Users can upload a Python function and invoke it — it runs in an isolated container and returns output. Both of you understand replication, chunking, and the Lambda execution model.

DAY 15 — Storage data models + file metadata index on master

PREREQUISITES BEFORE THIS DAY

Week 2 compute engine fully working and merged to main. Both understand task execution end-to-end.

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Extend shared/models.py with storage models: - FileChunk: chunk_id, file_id, chunk_index, size_bytes, checksum(md5), worker_id - FileMetadata: file_id, filename, total_size, chunk_count, replication_factor, created_at, status('available'/'degraded'/'lost') - UploadRequest: filename, content (base64 encoded bytes) - DownloadResponse: file_id, filename, content (base64) Create master/db/storage_models.py — ORM models for FileMetadata and FileChunk. Create master/db/storage_repository.py: - create_file_record(metadata: FileMetadata) - add_chunk(chunk: FileChunk) - get_file(file_id) -> (FileMetadata, list[FileChunk]) - get_chunks_by_worker(worker_id) -> list[FileChunk] - update_chunk_location(chunk_id, new_worker_id)	Create worker/storage/local_storage.py: - Base storage path: /tmp/minicloud-storage/{node_id}/ - save_chunk(chunk_id: str, data: bytes) -> path - read_chunk(chunk_id: str) -> bytes - delete_chunk(chunk_id: str) - get_chunk_checksum(chunk_id: str) -> str (md5 of stored bytes) - list_chunks() -> list[str] (all chunk_ids stored locally) - get_storage_stats() -> {used_bytes, available_bytes, chunk_count} Create worker/api/storage_routes.py: - POST /chunks — receives chunk_id + data, stores locally, returns checksum - GET /chunks/{chunk_id} — returns chunk bytes - DELETE /chunks/{chunk_id} — removes chunk - GET /chunks — lists all chunk_ids on this worker	*Person A owns the metadata (what exists, where it is). Person B owns the actual bytes (storing and retrieving them). These are completely independent today — they come together on Day 16. Branch: feat/A-storage-metadata / feat/B-chunk-storage*

END-OF-DAY SYNC AGENDA

1. Person B demos: manually POST a chunk to the worker API. GET it back. Verify the bytes match exactly. Check checksum

2. Person A demos: insert a FileMetadata record and two FileChunk records in the DB. Query them back
3. Discuss the replication strategy: file is split into chunks. Each chunk is sent to 2 different workers. If worker-1 dies, the chunk is still on worker-2. Draw this on paper
4. Discuss checksums: why do we compute an MD5 of each chunk? What attack or error does this catch? (Answer: bit rot, incomplete transfers, storage corruption)
5. Discuss: should we split every file into chunks, or only large files? What is the overhead of chunking a 1KB file? (Answer: chunking adds metadata overhead — for this project, chunk files above 1MB, send small files whole)

DAY 16 — File upload pipeline with 2x replication

PREREQUISITES BEFORE THIS DAY

Storage metadata DB working. Worker chunk API working. Both understand the chunk model.

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Create master/storage/storage_manager.py — the upload orchestrator: upload_file(filename, data: bytes): 1. Generate file_id (uuid) 2. Split data into 1MB chunks 3. For each chunk: a. Pick 2 healthy workers (not same worker twice) b. POST chunk to both workers via /chunks c. Verify returned checksum matches expected d. Save FileChunk records in DB (one per worker) 4. Save FileMetadata record 5. Return file_id download_file(file_id) -> bytes: 1. Get FileMetadata + FileChunks from DB 2. For each chunk_index (in order): a. Try primary worker first b. If worker dead → try replica worker 3. Verify each chunk checksum 4. Concatenate chunks in order 5. Return full bytes Create master/api/storage_routes.py: - POST /files (upload) - GET /files/{file_id} (download) - GET /files (list all files)	Implement re-replication in master/storage/replication_monitor.py: - Background task running every 60 seconds - Query DB: find FileChunks where worker_id is in dead-node list - For each under-replicated chunk: a. Find a healthy worker that has the surviving replica b. GET the chunk from that worker c. POST it to a new healthy worker d. Update FileChunk record in DB with new worker_id e. Log: 'Re-replicated chunk {id} from {old_worker} to {new_worker}' - Mark FileMetadata.status = 'degraded' if replication is in progress - Mark back to 'available' once all chunks have 2 replicas again	*This is the most satisfying integration to demo: 1. Upload a file, note file_id 2. Kill one worker that holds a chunk 3. Download the file — it should still work from the replica 4. Wait 60 seconds — re-replication should kick in 5. Start a new worker — the chunk migrates Branch: feat/A-storage-manager / feat/B-replication-monitor*

END-OF-DAY SYNC AGENDA

1. Live demo: upload a 5MB file, kill a worker, download the file — it still works. This is fault tolerance in action
2. Person A explains the chunk-splitting logic: how do you split bytes into 1MB pieces in Python? (bytes[0:1048576], bytes[1048576:2097152], etc.)
3. Person B explains the re-replication monitor: what triggers it? How does it know which chunks are under-replicated?
4. Discuss: What if BOTH workers holding a chunk die simultaneously? (Answer: file is lost unless replication factor is 3 — discuss the tradeoff)
5. Discuss: Why verify checksums on download and not just trust the stored data? (Answer: storage can corrupt silently over time — checksums detect this)

DAY 17 — Lambda function execution service

PREREQUISITES BEFORE THIS DAY

Storage with replication working. Both understand chunk upload/download/replication cycle.

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Create the Lambda orchestration on master: shared/models.py additions: - FunctionDef: function_id, name, code(str), runtime('python3.11'), created_at - InvokeRequest: function_id, payload(dict), timeout_seconds(int=30) - InvokeResult: function_id, output(str), error(str), duration_ms, worker_id master/functions/function_store.py: - Store function code as text in DB (use existing storage or new table) - upload_function(name, code) -> function_id - get_function(function_id) -> FunctionDef master/api/function_routes.py: - POST /functions (upload code as string) - POST /functions/{function_id}/invoke (trigger execution) - GET /functions/{function_id}/invocations (execution history)	Create worker/functions/function_runner.py — the Lambda executor: run_function(func: FunctionDef, payload: dict) -> InvokeResult: 1. Write func.code to a temp file: /tmp/fn_{id}.py 2. Inject payload as JSON string into the top of the file: PAYLOAD = {json.dumps(payload)} 3. Build Docker run command: docker run --rm \ -v /tmp/fn_{id}.py:/app/handler.py \ --memory=128m --cpus=0.25 \ --network=none \ --timeout=30 \ python:3.11-slim python /app/handler.py 4. Capture stdout as output, stderr as error 5. Clean up temp file 6. Return InvokeResult Create worker/api/function_routes.py: - POST /invoke — receives FunctionDef + payload, runs function_runner, returns result	*Lambda is conceptually simple: wrap user code in a container, run it, return output. Test with a simple function: code = 'import json; payload = PAYLOAD; print(json.dumps({"result": payload["x"] + payload["y"]}))' invoke with payload = {"x": 5, "y": 3} → should return {"result": 8} Branch: feat/A-function-routes / feat/B-function-runner*

END-OF-DAY SYNC AGENDA

1. Demo: upload a Python function that sorts a list and returns the sorted result. Invoke it with a random list. Verify output
2. Person B explains the Dockerfile-less execution: how does mounting a .py file into a python:3.11-slim container work? Why --network=none? Why --rm?
3. Person A explains: why does the master store function code in the DB instead of the storage service? (Answer: functions are small text — storage service is for large binary files)
4. Discuss security: right now user code can do anything (import os, rm -rf). How does network=none help? What else could you add? (cpus limit, read-only filesystem mount, no-new-privileges Docker flag)
5. Discuss cold start: the first invocation of a function is slow because Docker pulls the python:3.11-slim image. How does AWS Lambda solve this? (Answer: container reuse — they keep warm containers)

DAYS 18–19 — Storage + Lambda integration tests + Week 3 buffer

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Write tests/test_storage.py: - Upload 1KB, 1MB, and 10MB files - Verify downloaded bytes match uploaded bytes exactly - Test worker death during upload (mock) - Test re-replication trigger - Test corrupted chunk detection (modify a chunk on disk, try to download) Also: add storage stats to GET /nodes response — each worker reports disk usage in heartbeat	Write tests/test_lambda.py: - Test math function: add two numbers - Test error handling: function raises an exception — verify error captured, not crash - Test timeout: function sleeps 60 seconds — verify it is killed after 30s - Test payload injection: verify PAYLOAD variable is accessible in user code - Test concurrent invocations: 5 simultaneous invokes on same function	*Run all tests on actual hardware. Aim: 100% of tests pass before Day 20. Branch: feat/A-storage-tests / feat/B-lambda-tests → merge to dev → main*

DAYS 20–21 — Week 3 Knowledge Sync + Buffer

WEEK 3 UNDERSTANDING TOPICS

Storage deep dive: draw the full upload flow on paper. Where does each chunk go? What DB records are created? What HTTP calls happen? Lambda deep dive: trace a function invocation. What happens from POST /invoke to the Python process running inside Docker? Fault tolerance: physically kill a worker mid-upload. What happens? Are there any stuck states? Fix them. Buffer: any features from Week 1–3 that are incomplete or fragile should be hardened now.

WEEK 4 — MONITORING, HARDENING & FULL SYSTEM INTEGRATION (Days 22–28)

WEEK 4 GOAL

By end of Week 4: Prometheus + Grafana dashboards showing real-time CPU, RAM, task counts, and storage usage for all 4 nodes. Centralized structured logs queryable by task_id. Full stress test passes (50 concurrent tasks, worker process deaths, WiFi disconnection simulation). A complete README that lets any engineer run your system from scratch.

DAYS 22–23 — Prometheus metrics endpoint on all nodes

PREREQUISITES BEFORE THIS DAY

Install on all machines: pip install prometheus-client. Prometheus server on master: docker run -p 9090:9090 prom/prometheus with a config file pointing to all worker /metrics endpoints.

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Add Prometheus metrics to master/api/metrics_routes.py: - GET /metrics endpoint (Prometheus text format) - Expose these gauges: * minicloud_healthy_nodes (count of healthy nodes) * minicloud_total_tasks_submitted (counter) * minicloud_tasks_by_status{status=...} (gauge per status) * minicloud_files_stored (gauge) * minicloud_total_storage_bytes (gauge) * minicloud_scheduler_decisions_total (counter) Use prometheus_client library: from prometheus_client import Gauge, Counter, generate_latest HEALTHY_NODES = Gauge('minicloud_healthy_nodes', 'Number of healthy worker nodes') # Update in health_monitor: HEALTHY_NODES.set(len(get_healthy_nodes()))	Add Prometheus metrics to worker/api/metrics_routes.py: - GET /metrics endpoint on each worker - Expose: * minicloud_worker_cpu_percent (gauge, updated from psutil) * minicloud_worker_ram_percent (gauge) * minicloud_worker_active_tasks (gauge) * minicloud_worker_tasks_completed_total (counter) * minicloud_worker_tasks_failed_total (counter) * minicloud_worker_storage_used_bytes (gauge) * minicloud_worker_chunks_stored (gauge) Configure Prometheus scrape config (prometheus.yml): scrape_configs: - job_name: 'minicloud-master' static_configs: - targets: ['192.168.1.10:8000'] - job_name: 'minicloud-workers' static_configs: - targets: ['192.168.1.11:8001', '192.168.1.12:8001', '192.168.1.13:8001']	*Both work independently, then combine in Prometheus config. After setup, visit http://master:9090 — type 'minicloud_healthy_nodes' in the query box — you should see a value. Branch: feat/A-master-metrics / feat/B-worker-metrics Verify: Prometheus scrapes all 4 targets with no errors (Status → Targets page in Prometheus UI)*

END-OF-DAY SYNC AGENDA

1. Open Prometheus UI together. Run queries: minicloud_worker_cpu_percent — see all 3 workers. minicloud_tasks_by_status{status='COMPLETED'} — see total completed

2. Discuss: what is a Counter vs a Gauge in Prometheus? Why can't you use a Gauge for total tasks completed? (Answer: Gauges go up and down; Counters only go up — tasks_completed is monotonically increasing)
3. Discuss: Prometheus pulls metrics from nodes (scraping). Compared to nodes pushing metrics, what are the advantages? (Answer: scraper controls the cadence; failed scrape means the node is down — built-in health check)
4. Deliberately spike CPU on a worker (run a heavy script) — watch the CPU gauge in Prometheus update in real time

DAYS 24–25 — Grafana dashboards + centralized log aggregation

PREREQUISITES BEFORE THIS DAY

Prometheus collecting metrics from all nodes. All metrics accessible in Prometheus query UI.

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Set up Grafana and build master dashboard: - docker run -p 3000:3000 grafana/grafana - Add Prometheus as data source (http://master:9090) - Build 'Cluster Overview' dashboard with panels: * Healthy nodes count (stat panel) * Tasks submitted today (stat) * Active tasks right now (gauge) * Tasks by status (pie chart) * Files stored + total bytes (stat) * Time series: tasks completed over time - Export dashboard JSON to infra/grafana/cluster-overview.json	Build worker dashboards + centralized log viewer: - Grafana 'Worker Nodes' dashboard: * CPU% per worker (time series, all 3 on same chart) * RAM% per worker (same) * Active tasks per worker (bar chart) * Storage used per worker - Centralized log endpoint on master: * POST /logs — workers send JSON log lines here * Master stores in DB table: logs(id, timestamp, node_id, level, component, event, message) * GET /logs?node_id=X&level=ERROR&limit=100 — queryable API - Update worker logger to also POST to master /logs (fire-and-forget)	*Person A owns the master-level view. Person B owns the per-worker view. At EOD: open both dashboards side by side. Submit 10 tasks and watch the charts update in real time. Export both dashboards as JSON to the repo — anyone who clones the repo can import them. Branch: feat/A-grafana-cluster / feat/B-grafana-workers-logs*

END-OF-DAY SYNC AGENDA

1. Both dashboards open. Submit 20 tasks across 3 workers. Watch CPU, active tasks, completion counter update live
2. Query the log API: GET /logs?level=ERROR — show any errors that occurred today. Can you trace a failed task from its task_id in the logs?
3. Person A explains Grafana panel types — when would you use a time series vs a stat vs a gauge panel?
4. Person B explains the log aggregation model — why fire-and-forget (not waiting for log POST to complete)?
5. Discuss: What would you add to these dashboards if this were production? (Alerts when CPU > 80%, p99 task latency, error rate trends)

DAYS 26–27 — Full system stress test + fault injection + Redis Streams upgrade

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Write tests/test_stress.py — the final boss: - 50 concurrent tasks submitted simultaneously - Mix of: compute tasks, file uploads/downloads, lambda invocations - Verify: all 50 eventually complete or fail with clear error (nothing stuck forever) - Verify: no task_id appears twice in results - Verify: active_tasks counter in heartbeats returns to 0 after all complete Fault injection tests: - Kill worker during task execution → task reaper reschedules - Kill worker during file upload → re-replication kicks in - Kill master for 30 seconds, restart → Redis restores node registry, pending tasks resume - Saturate one worker (manual CPU spike) → scheduler routes away from it	Upgrade task dispatch to Redis Streams (optional but highly recommended): - Instead of master directly POSTing to worker, publish task to Redis Stream: XADD tasks * task_id {id} payload {json} - Each worker runs a consumer group: XREADGROUP GROUP workers worker-{id} COUNT 1 BLOCK 5000 STREAMS tasks > - Worker processes task, ACKs it: XACK tasks workers {message_id} - Master polls task status from DB instead of waiting for HTTP callback - Benefit: if a worker dies mid-read, the message stays unACKed and another worker picks it up automatically	*Stress tests are shared — both run them and compare results. Redis Streams is optional — if time is short, skip to documentation. Branch: feat/A-stress-tests / feat/B-redis-streams Goal: all stress tests pass with Redis Streams enabled*

END-OF-DAY SYNC AGENDA

1. Run stress test. Review failures. Categorize: are failures in the task code (expected), in our system (bugs), or in resource limits (expected)?
2. If Redis Streams implemented: Person B explains XADD, XREADGROUP, XACK, consumer groups. What is the difference between consumer group and pub/sub?
3. Discuss: in the stress test, what was the bottleneck? Master CPU? Worker Docker startup time? Network? DB writes? This analysis is gold for any system design interview
4. Review: are there any stuck states left? Tasks that stay RUNNING forever, chunks that never get re-replicated, log POSTs that fail silently?

DAY 28 — Final README, architecture diagram, and project retrospective

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Write the master README.md: 1. Project overview (3 sentences) 2. Architecture diagram (ASCII or link to image) 3.	Write the architecture decision record (docs/ARCHITECTURE.md): For each major decision, document: - What was chosen	*Final PR: both documents merged into main. Tag the release: git tag v1.0.0 Final repo state: anyone who clones it should be able to run the whole system. Celebrate*

YOU (Person A)	FRIEND (Person B)	HOW THEY MERGE ON GITHUB
Prerequisites: exact versions of Python, Docker, Redis, PostgreSQL 4. Setup: step-by-step for all 4 laptops (WiFi network config, static IP assignment via router DHCP reservation, installs) 5. Running the system: start order (master first, then workers) 6. API reference: every endpoint with example curl commands 7. Running tests: how to run unit tests and integration tests 8. Known limitations and future improvements (note: tested on WiFi, wired Ethernet gives better stress test results) This README is your portfolio artifact. A stranger should be able to run your system from scratch.	- What alternatives were considered - Why this choice was made - What tradeoffs were accepted Decisions to document: - FastAPI over Flask/gRPC - Resource-aware scheduling over round-robin - Replication over erasure coding - Redis for registry over pure in-memory - REST over message queue (or Redis Streams if implemented) - PostgreSQL for task metadata over MongoDB This document demonstrates engineering maturity — you understand WHY, not just HOW.	*— you built a distributed cloud platform from scratch.*

END-OF-DAY SYNC AGENDA
1. Retrospective: what was the hardest part? What would you do differently?
2. Each person presents the system to the other as if presenting to a senior engineer — explain every architectural decision
3. List 5 things you would add if you had another week (Kafka, Kubernetes-style resource reservations, multi-tenancy, TLS, CLI client)
4. Review each other's documentation — is it clear enough for a stranger? Are all curl examples correct?
5. Discuss: how does this compare to what AWS actually does under the hood? What parts did you simulate, what parts are fundamentally different?

APPENDIX — Quick Reference Cards

DAILY MORNING RITUAL (5 min)	DAILY EVENING RITUAL (5 min)
git checkout dev git pull origin dev git checkout -b feat/A-{today-feature} Check: all 4 machines reachable over WiFi (ping each) Check: Redis, PostgreSQL running on master	git add . && git commit -m 'feat: description' git push origin feat/A-{feature} Open PR on GitHub into dev Review friend's PR (20 min) Sync call: walk through each other's code

4-WEEK OVERVIEW AT A GLANCE

Week	Theme	You Build	Friend Builds	End State
1	**Foundation**	Node Registry, Redis persistence, health monitor, scheduler lock	Worker registrar, heartbeat sender, structured logging, task queue	*Nodes register, heartbeats flow, dead detection works*
2	**Compute**	Task DB + ORM, scheduler algorithm, task dispatch, task reaper	Docker executor, task queue, concurrency semaphore, timeout handling	*Full task lifecycle: submit → schedule → run → result*
3	**Storage + Lambda**	File metadata, upload orchestrator, re-replication monitor, function store	Chunk storage API, local file handler, function runner, Docker mount	*Files survive worker death; functions run in isolation*
4	**Observability**	Prometheus master metrics, Grafana cluster dashboard, stress tests, README	Prometheus worker metrics, log aggregation, Grafana worker dashboard, ARCHITECTURE.md	*Full observability, documented system, stress-tested*